

FarmGuard

IoT-Based Intelligent Farm Decision & Waste Detection System

IoTrix 2.0 Semi-Final: Project Proposal

Competition Track	Track A: Embedded IoT System Development
Team	T4D
Team Members	M.M.Bin Mushan M.Afnan K. Jelaxsi J. Kamsa
University	South Eastern University of Sri Lanka
GitHub Repository	https://github.com/BinMushan/FarmGuard
Submission	Semi-Final submission, 5 October 2026

1. Problem Statement

Many smart farming systems measure soil moisture, temperature, humidity, pH and NPK, and show these values on an app or dashboard. These readings are useful, but they do not tell a farmer whether a specific action is needed right now. Farmers often irrigate or apply fertilizer because of routine, habit or incomplete information, even when the soil or the weather may not call for it. This can lead to wasted water, wasted fertilizer and extra cost, and in some cases it can harm the crop or the environment.

The target users are small and medium-scale farmers who grow field or vegetable crops and make regular decisions about irrigation, fertilizer and spraying. The use case is simple: before the farmer irrigates, fertilizes or sprays, FarmGuard checks the planned action against what the field sensors and the weather forecast are showing, and gives a clear recommendation.

2. Existing Gap and Motivation

Most IoT farming systems stop at monitoring. They answer the question "what is happening in the field?" but not "should I do this action now?". The farmer still has to read the numbers alone, and the system has no idea what the farmer is planning to do.

FarmGuard focuses on that second question. It takes the farmer's intended action and evaluates it using live sensor data, crop information and the weather forecast. The aim is decision support, not replacing the farmer's judgement.

3. Proposed Solution

FarmGuard is an ESP32-based IoT system. Sensors in the field send soil and air data to the ESP32, which checks the data, shows a quick local result, and sends everything to a cloud backend. When the farmer enters a planned action (irrigation, fertilizer or pesticide spray), a rule-based decision engine checks it against the latest sensor readings, the crop type and growth stage, and the weather forecast. The action is classified as:

- **Necessary** (green LED): the field conditions support the action.
- **Risky** (yellow LED): the action may be badly timed or may cause a problem, for example fertilizer before heavy rain.

- **Potentially Unnecessary** (red LED and buzzer): the conditions suggest the action may not be needed at this moment.

If an action is potentially unnecessary, the system gives an estimate of the water or fertilizer amount and the approximate cost that could be avoided. An LLM/AI service then explains the result in simple language, and important alerts are sent to the farmer through WhatsApp and a web dashboard.

The decision is made by the rule-based engine. The LLM is only used to explain the result and is never part of the decision itself.

4. Objectives

- Build an ESP32 prototype that reads soil moisture, soil temperature, pH, NPK, air temperature and humidity.
- Let the farmer select a planned action (irrigate, fertilize, spray) and show a local verdict on an OLED display, LEDs and a buzzer, even without internet.
- Send sensor data reliably to a cloud backend and store it with timestamps.
- Build a transparent decision engine that uses sensors, crop information and the weather forecast to classify the planned action.
- Estimate the amount and cost that could be avoided when an action seems unnecessary.
- Explain every recommendation in simple language and deliver alerts through a dashboard and WhatsApp.
- Test the prototype under changing input conditions and record the results honestly.

5. System Architecture

The system has four parts. The field layer contains the sensors and the ESP32. The network layer carries the data over Wi-Fi. The cloud and intelligence layer receives, stores and evaluates the data, using external services such as the weather API. The farmer interface shows the result and the explanation. Figure 1 shows the planned data flow.

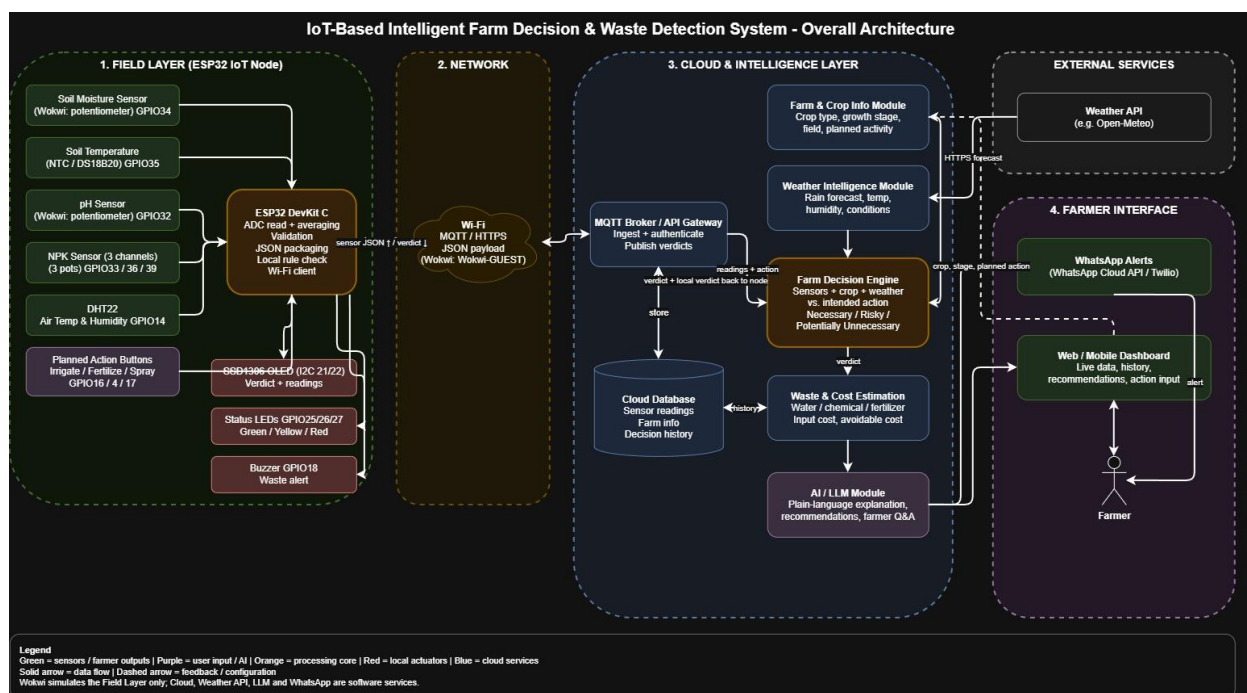


Figure 1: Overall system architecture of FarmGuard

System workflow

Farm sensors → ESP32 (read, average, validate, local rule check) → Wi-Fi (MQTT / HTTPS, JSON) → MQTT broker / API gateway → Farm Decision Engine (with weather and crop information) → classification → waste and cost estimate → LLM explanation → dashboard and WhatsApp alert. The verdict is also sent back to the ESP32 so that the local display shows the final result.

Stage	Purpose
Farm sensors and buttons	Measure the real soil and air conditions and let the farmer select the planned action.
ESP32 and firmware	Read and average the sensors, check that the values make sense, run a simple local rule check, package the data as JSON and send it. This is the core of the system.
Wi-Fi and MQTT / HTTPS	Carry readings to the cloud and the verdict back to the node.
MQTT broker / API gateway	Receive and authenticate the data, and publish verdicts back.
Cloud database	Store sensor readings, farm and crop details, and decision history.
Farm and crop info module	Keep the crop type, growth stage, field and planned activity.
Weather intelligence module	Bring in the rain forecast, temperature, humidity and conditions from a weather API.
Farm decision engine	Compare the planned action with sensors, crop and weather and give the verdict.
Waste and cost estimation	Estimate the water, fertilizer or chemical amount and the cost that could be avoided.
AI / LLM module	Turn the result into a short explanation and answer simple farmer questions.
Dashboard and WhatsApp	Show live data, history and recommendations, and send alerts.

6. Track A Focus: The Physical IoT System

FarmGuard is built around the embedded system. The ESP32 and the sensors collect the soil and air data, and the firmware is responsible for reading, validating, deciding locally and sending that data. The cloud, weather API, LLM and WhatsApp services extend the embedded system but do not replace it. The node still gives a local verdict on its own display, LEDs and buzzer if the network is lost.

If the input conditions change, for example when dry soil is watered, the sensor readings change, and the verdict is expected to change with them. This is the main behaviour we plan to demonstrate in the live session, on real hardware, and we also check it in simulation under different conditions (Section 14).

7. Hardware and Firmware

7.1 Field node circuit

The field node circuit has been designed and wired in the Wokwi simulator. Figure 2 shows the circuit and Table 1 gives the pin map. The circuit is the Field Layer of the architecture. Wokwi

simulates the field layer only, and the cloud, weather API, LLM and WhatsApp are software services.

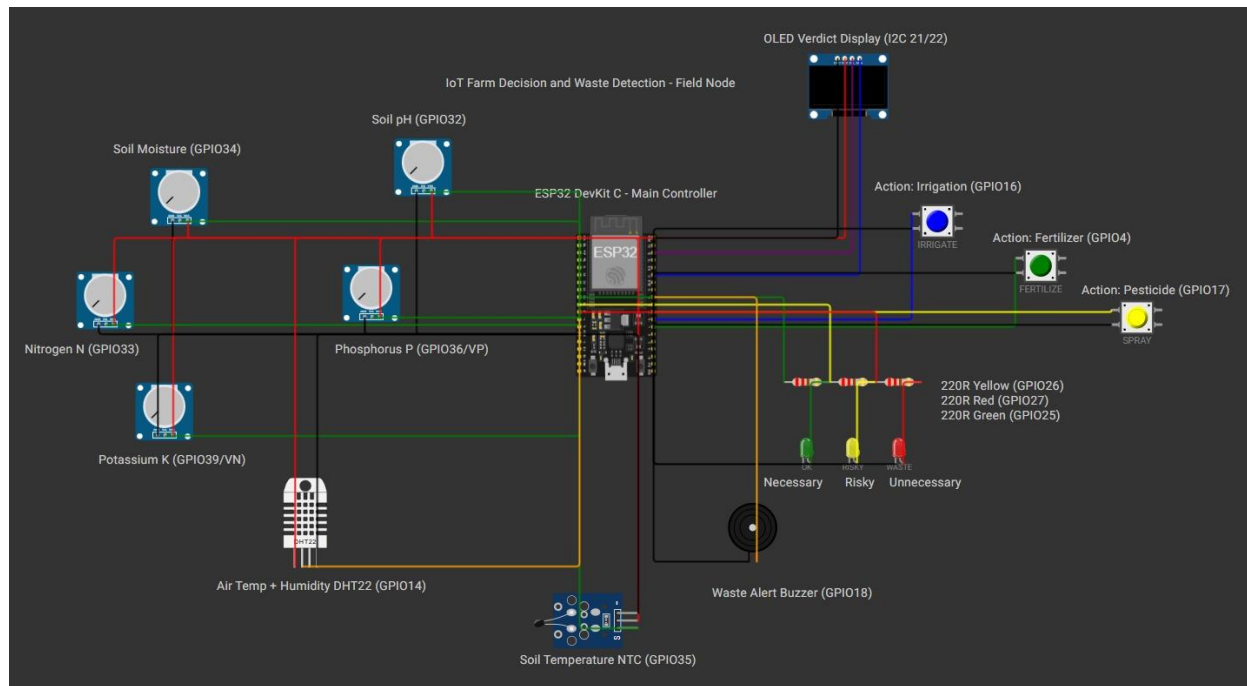


Figure 2: Field node circuit (ESP32 DevKit C) in Wokwi

Table 1: ESP32 pin map

Component	ESP32 pin	Role
Soil moisture sensor	GPIO34	Analog input
Soil pH sensor	GPIO32	Analog input
Nitrogen (N)	GPIO33	Analog input
Phosphorus (P)	GPIO36 (VP)	Analog input
Potassium (K)	GPIO39 (VN)	Analog input
Soil temperature (NTC)	GPIO35	Analog input
Air temperature and humidity (DHT22)	GPIO14	Digital data
OLED verdict display (SSD1306)	GPIO21 (SDA), GPIO22 (SCL)	I2C
Action button: Irrigation	GPIO16	Digital input
Action button: Fertilizer	GPIO4	Digital input
Action button: Pesticide / spray	GPIO17	Digital input
Green LED (Necessary)	GPIO25	Output via 220 Ω
Yellow LED (Risky)	GPIO26	Output via 220 Ω
Red LED (Potentially Unnecessary)	GPIO27	Output via 220 Ω
Waste alert buzzer	GPIO18	Output

All six analog sensors are on ADC1 pins (GPIO32 to 39). This was chosen on purpose, because the ADC2 pins cannot be used reliably while Wi-Fi is on.

7.2 Hardware components

Function	In the simulation (Wokwi)	Planned real hardware
Main controller	ESP32 DevKit C	ESP32 DevKit C (built-in Wi-Fi)
Soil moisture	Potentiometer	Soil moisture sensor
Soil pH	Potentiometer	Analog pH sensor / probe
NPK	Three potentiometers	NPK sensor (usually RS485)
Soil temperature	NTC sensor	NTC or DS18B20 probe
Air temp / humidity	DHT22	DHT22
Farmer input	3 push buttons	3 push buttons
Local output	OLED, 3 LEDs, buzzer	OLED, 3 LEDs, buzzer

In the simulation, the potentiometers stand in for the moisture, pH and NPK sensors so that we can change the input values easily. Real NPK sensors usually use an RS485 interface, so an RS485-to-TTL module and a UART connection will be needed on the real hardware, and the NPK part of the circuit will change at that point. The final wiring will be confirmed against the sensors we actually use.

7.3 Firmware responsibilities

- Read each sensor at a fixed interval, take an average of several samples, and convert the raw values to readable units.
- Apply calibration values and basic checks, such as rejecting readings outside a possible range.
- Detect which action the farmer selected (irrigate, fertilize or spray) from the three buttons.
- Run a simple local rule check and show the verdict on the OLED, the LEDs and the buzzer.
- Connect to Wi-Fi and send the readings and selected action as JSON, with a timestamp and device ID.
- Receive the cloud verdict and show it on the local display.
- Reconnect automatically if Wi-Fi or the broker connection drops, and keep the local verdict working while offline.
- Keep recent readings in memory while offline and send them when the connection returns

8. Software, Cloud and API Components

Part	Plan
Communication	Wi-Fi. MQTT is the preferred protocol because it is lightweight and suits small, frequent sensor messages. HTTPS is the alternative path through the API gateway. The payload is JSON. MQTT is the confirmed primary communication protocol.
Cloud backend	The existing AgriMind AI backend will be extended using Python and FastAPI to receive FarmGuard sensor data, authenticate the ESP32 device, store readings and run the FarmGuard decision engine.
Database	Stores sensor readings, farm and crop details, and decision history. MongoDB Atlas will be used as the cloud database.

Part	Plan
Web dashboard	The AgriMind AI platform displays live sensor readings, historical data, decision results, estimated waste and costs, and alerts, while allowing farmers to enter the crop type, growth stage, and planned agricultural action.
Weather API	Provides rain forecast, temperature, humidity, and general weather conditions over HTTPS using the Open-Meteo Weather API.
LLM / AI API	Explains the decision in simple, farmer-friendly language and answers farmer questions using the Gemini AI API integrated with AgriMind AI.
WhatsApp service	Sends decision alerts and important notifications to the farmer through the Meta WhatsApp Cloud API integrated with the AgriMind AI backend.

9. Decision-Making Approach

The decision engine uses clear rules that depend on the crop and its growth stage. Each crop and stage has a target range for soil moisture and nutrients, and the engine compares the live readings and the weather forecast with that range. Because the rules are written out, we can always show why a particular result was given.

The ESP32 runs a smaller version of these rules using only its own sensors, so that it can give a local verdict at any time. The cloud engine adds the weather forecast, crop stage and history, and its verdict is the final one when the network is available.

Example: irrigation

Result	Example condition
Necessary	Soil moisture is below the target range for the crop's growth stage, and no useful rain is forecast soon.
Potentially Unnecessary	Soil moisture is already within the target range, or enough rain is forecast soon to cover the need.
Risky	Irrigation is planned during heavy rain, or the soil is already close to saturated, or the readings are old or look unreliable.

Example: fertilizer and pesticide

For fertilizer, the engine looks at the NPK and pH readings, the growth stage and the rain forecast. Applying fertilizer just before heavy rain may be marked as Risky because of possible wash-off, and a nutrient that is already within range for that stage may be marked as Potentially Unnecessary. For pesticide spraying, the engine mainly checks the rain forecast and the air conditions, because spraying before rain can wash the chemical away and waste it.

The threshold values will be taken from reliable agricultural references for the selected crop and adjusted during testing. The prototype crop will be selected based on the availability of suitable agricultural reference data and testing conditions. The thresholds will be validated during prototype testing, ideally with guidance from an agricultural expert. The system provides decision support only and does not claim to scientifically prove that an agricultural action is or is not required.

10. Waste and Cost Estimation

When an action is classified as Potentially Unnecessary, the system gives a rough estimate of what could be saved. The estimate uses values entered by the farmer, such as field area, planned water or fertilizer amount, and local input prices.

- Irrigation: estimated water volume = field area × planned water depth, multiplied by the unit price of water if the farmer has a cost for it.
- Fertilizer: estimated amount = planned quantity per area × field area, multiplied by the fertilizer price.
- Pesticide: estimated amount = planned quantity per area × field area, multiplied by the chemical price.

The result is shown as an approximate avoidable amount and cost. It is an estimate based on the farmer's inputs and is not a measured saving. We have not yet measured real savings in a field.

11. AI/LLM Integration

The LLM receives the decision result, the key readings and the reason from the rule engine, and writes a short explanation in simple language. For example, it can explain that the soil already has enough moisture and rain is expected tonight. The farmer can also ask follow-up questions about the recommendation. The LLM does not change the classification. Its output is checked against the data given to it, and the dashboard always shows the original rule-based result next to the explanation. If the LLM service is not available, the rule-based result is still shown.

12. Technical Reasoning and Design Choices

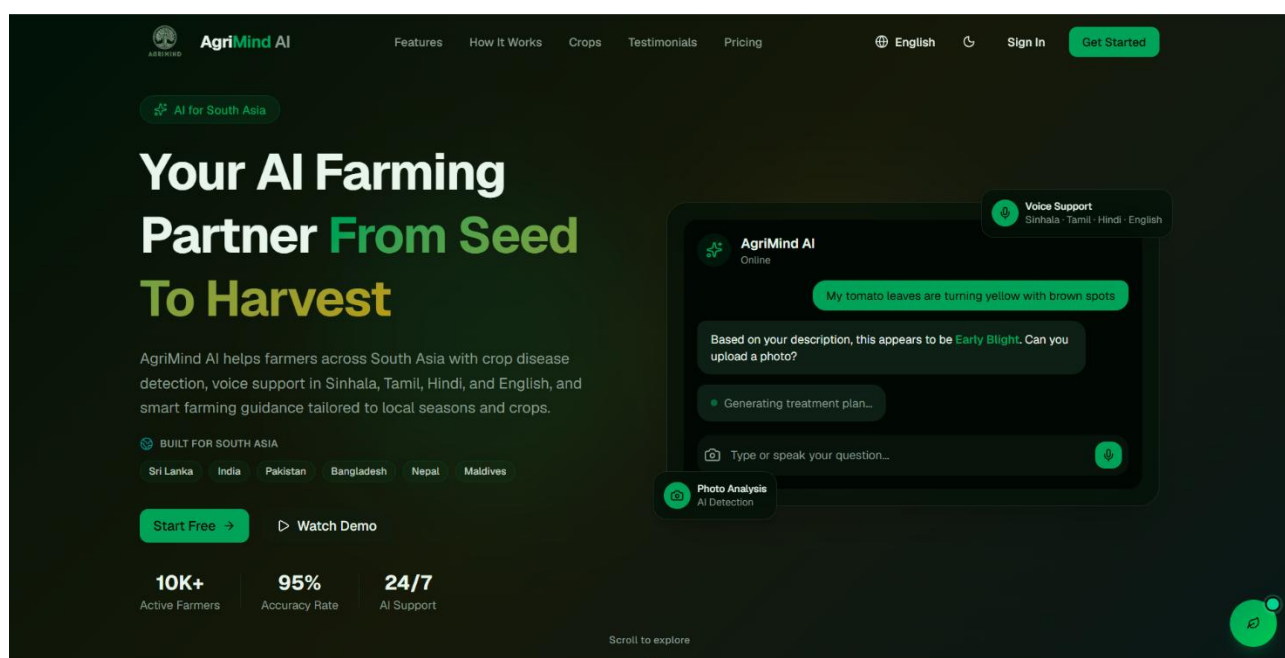
Question	Our reasoning
Why ESP32?	It has built-in Wi-Fi, enough ADC1 channels for all analog sensors, I2C for the display, and a low cost. It is also supported well in Wokwi, so we could design and test the circuit early.
Why MQTT?	MQTT messages are small and light, which suits frequent sensor data, and the broker can also publish the verdict back to the node. HTTPS is kept as a fallback.
Why not a simple dashboard only?	A dashboard alone only shows numbers. Our system also takes the planned action and gives a verdict, which is the gap we want to fill.
Why rules and not AI for the decision?	Rules are transparent, easy to explain to a farmer and easy to test. An AI model could give a wrong answer with no clear reason. So the LLM only explains.
What if the network is lost?	The ESP32 keeps reading and gives the local verdict on its OLED, LEDs and buzzer. It reconnects automatically. The cloud marks the data as old, and decisions are not made from old data.
Highest technical risk	The accuracy of low-cost NPK and pH sensors, because they drift and are sensitive to soil contact.
What fails in a real deployment?	Sensor drift, moisture or heat damage to the node, power loss, and Wi-Fi coverage in the field. These are handled by calibration, data checks, a protective enclosure and the offline local verdict.

13. Current Implementation / Proof of Concept

The table below separates what is done from what is still planned. Nothing is claimed as working unless we can show evidence for it.

Item	Status	Evidence
System architecture design	Done	Figure 1
Field node circuit and pin map	Done in Wokwi	Figure 2, Table 1, https://wokwi.com/projects/477027652387547137
ESP32 firmware: sensor reading, local verdict, OLED / LED / buzzer	In Progress	https://github.com/BinMushan/FarmGuard.git
WhatsApp alerts	Planned	
Web dashboard	In Progress	Figure 5

Figure 5 :



Parts that are not yet working are treated as planned work for the final stage.

14. Simulation Validity and Real-World Validation

We know that a simulation on its own is not enough proof for Track A. So we use the Wokwi model for two things only: to show the logic behaves correctly under many different conditions, and to compare it against the real ESP32 hardware. The simulation is never used alone as evidence. Every scenario below is also repeated on real hardware, and the real results are what we rely on. No measured results are reported in this proposal yet. The results columns are empty on purpose and will be filled only with what we actually observe.

14.1 Simulation demo under different conditions

In Wokwi we change the potentiometers (moisture, pH, N, P, K), the DHT22 and NTC values and the action buttons, and check the OLED, LED and buzzer output against the expected verdict from our rules. The expected verdicts below come from our design rules, and the thresholds are still to be tuned.

Table 2: Scenario test matrix

ID	Condition (input change)	Action	Expected verdict
S1	Dry soil, no rain	Irrigate	Necessary (green)
S2	Soil moisture already in target range	Irrigate	Potentially Unnecessary (red + buzzer)
S3	Soil near saturated	Irrigate	Risky (yellow)
S4	N, P and K all within range	Fertilize	Potentially Unnecessary
S5	Low nitrogen, no rain	Fertilize	Necessary
S6	Low nitrogen, heavy rain in forecast (cloud input)	Fertilize	Risky
S7	Rain in forecast or very high humidity	Spray	Risky
S8	Impossible or disconnected sensor value	Any	Value rejected, flagged, no normal verdict
S9	Wi-Fi switched off	Any	Local verdict still shown, node reconnects later

Scenario S6 uses the weather forecast, so it is tested by feeding a forecast value into the cloud engine. The other scenarios use only the sensors on the node.

14.2 Real-world validation

To show that the simulation is valid in real scenarios, we run the same firmware logic on a physical ESP32 with real sensors and check the real readings against simple reference measurements. We will use the sensors we can get working properly, with soil moisture, air temperature and humidity and soil temperature as the first priority. NPK and pH are treated as indicative because low-cost versions are less reliable.

Table 3: Real-world validation plan

What is checked	How	Pass idea
Soil moisture response	Put the real sensor in the same soil at dry, damp and wet states. Check the weight-based moisture (weigh, dry, weigh again) or a reference moisture meter if we can get one.	Readings rise and fall in the right order and roughly match the reference.
Air temperature and humidity	Compare the DHT22 with a reference thermometer / hygrometer.	Readings stay close to the reference.
Soil temperature	Compare with a reference thermometer in the same soil.	Readings stay close to the reference.
pH	Two-point check in pH 4 and pH 7 buffer solutions, then a soil sample.	Readings separate the buffers correctly after calibration.
NPK	Compare soil with and without added fertilizer, or known fertilizer solutions.	Readings move in the expected direction (indication only).
Real scenario (physical test)	Water dry soil in a pot and watch the live reading and the verdict change, for example from Necessary to Potentially Unnecessary.	Reading and verdict both change as expected.
Simulation vs hardware match	Log the real readings, replay the same values in Wokwi, and compare the verdicts from both.	Both give the same verdict for the same inputs.

We will judge validity by three things: the Wokwi verdict and the real-hardware verdict should match for the same inputs, the real readings should stay close to the reference measurements,

and the reading and verdict should change in the correct direction when a real condition changes. If the hardware and the simulation disagree, we will report it and fix the cause. For the live demonstration, the main proof is the real hardware test and the comparison with the simulation.

15. Limitations and Risks

Limitation / risk	Description and handling
Sensor accuracy	Low-cost NPK and pH sensors can be unreliable and drift over time. We will calibrate, use readings as indications rather than exact values, and flag suspicious data. NPK sensing is the highest technical risk.
Simulation vs real hardware	The current circuit is in Wokwi, where potentiometers replace some sensors, so a simulation alone does not prove the system works. We validate it with the real-hardware checks in Section 14. Real sensors, especially RS485 NPK, need extra interfacing and calibration work.
Soil and placement variation	One sensor point may not represent a whole field. The prototype covers a single point only.
Rule thresholds	Thresholds depend on crop, soil and region, and are still to be validated.
Network loss	If Wi-Fi is lost, the ESP32 keeps reading, gives a local verdict and tries to reconnect. The cloud shows the data as old, and decisions are not made from old data.
Field conditions	Outdoor use needs waterproofing, stable power and protection from heat. The prototype is not yet field-hardened.
External services	The weather API, LLM and WhatsApp service may be unavailable or have usage limits. The rule-based result is still shown without the explanation or the forecast when they fail.
Cost estimates	Estimates depend on farmer-entered values and are approximate.
LLM wording	The LLM may phrase things wrongly, so it only explains rule output and the original result is always shown.

16. Planned Improvements for the Final

- Move from the simulated circuit to a physical prototype with real sensors, and complete and test all firmware features, including offline buffering.
- Complete the cloud backend, decision engine, cost estimation, dashboard, WhatsApp alerts and LLM explanations, and connect them end to end.
- Complete the simulation scenario tests and the real-world validation (Section 14) and report the results, including any mismatch.
- Calibrate sensors and validate thresholds for the chosen crop, with expert advice if possible.
- Test the prototype on real soil for a longer period and report the results.
- Improve the enclosure, power supply and weather protection for outdoor use.

Future improvements beyond the final competition may include connecting an irrigation valve through a relay, using more sensor nodes per field, and learning from the stored decision history to improve the rules.

17. Technology Stack Summary

Area	Technology
Hardware	ESP32 DevKit C, soil moisture, soil temperature (NTC), pH and NPK sensors, DHT22, SSD1306 OLED, LEDs, buzzer, push buttons
Firmware	Arduino framework using C++ for ESP32 sensor interfacing, local decision logic, OLED display, LED/buzzer control, and Wi-Fi communication.
Simulation	Wokwi (field layer only)
Connectivity	Wi-Fi, MQTT and/or HTTPS, JSON payload
Cloud and software	MQTT broker / API gateway, cloud backend, database, rule-based decision engine, web dashboard
External services	Weather API, LLM/AI API, WhatsApp API/service
Project workspace	GitHub: BinMushan/FarmGuard